

Automedon Mobile API — Developer Guide

API reference for the **read-only** companion mobile app (Flutter).

- The app can **log in, view** cars / availability / reservations / Vision360, and receive **notifications**.
 - The app **cannot create, update, or delete** any business data. The only write it performs is registering its own push token.
 - Backend: Supabase Edge Functions. Each endpoint is an independent function.
-

1. Base URL

```
https://<PROJECT_REF>.supabase.co/functions/v1
```

Replace <PROJECT_REF> with the Supabase project ref you are given (e.g. abcdefghijklmno). Every path below is relative to this base URL.

Ask the backend owner for the exact <PROJECT_REF> and a test tenant slug/username/password.

2. Authentication

Auth is **token-based**:

1. The app calls POST /mobile-auth with slug + username + password.
2. It receives a **bearer token** (JWT, valid **30 days**).
3. Every other request must send:

```
Authorization: Bearer <token>
```

Notes:

- No Supabase `apikey` header is required — send only the `Authorization` header.
 - Store the token securely (e.g. `flutter_secure_storage`).
 - When any endpoint returns **401**, treat the session as expired → send the user back to login and request a new token.
 - `slug` identifies the tenant (the company). It is the same value used in the web app URL, e.g. `Cherif-Rent-Car`.
-

3. Conventions

Headers (authenticated requests)

```
Authorization: Bearer <token>
Content-Type: application/json
```

Success: HTTP 200 + JSON body (shapes documented per endpoint).

Error: non-2xx + JSON:

```
{ "error": "Human-readable message" }
```

Status	Meaning
400	Bad request (missing/invalid params or body)
401	Missing / invalid / expired token, or bad login
403	Forbidden
404	Resource not found (or not visible to this user)
405	Wrong HTTP method
500	Server error

Dates: ISO 8601 strings (e.g. 2026-06-30T14:00:00.000Z). Due/expiry dates for papers are YYYY-MM-DD.

Money: numbers in the tenant's currency (TND). No currency formatting applied.

Roles: admin, sub_office, assistant, user. A sub_office user only sees the cars assigned to them and the reservations they created — the API enforces this automatically; the app doesn't need to do anything special.

4. Endpoints

4.1 POST /mobile-auth — Login

Public (no token). Verifies credentials and returns a token.

Request body

```
{
  "slug": "Cherif-Rent-Car",
  "username": "admin",
  "password": "secret"
}
```

200 response

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "0b3f...uuid",
    "username": "admin",
    "role": "admin",
    "allowed_pages": null
  },
  "tenant": {
```

```

    "id": "a1c2...uuid",
    "slug": "Cherif-Rent-Car",
    "name": "Cherif Rent Car",
    "logo_url": "https://.../logo.png",
    "status": "active"
  }
}

```

Errors: 400 missing fields / bad JSON, 401 invalid credentials.

- `allowed_pages` is either `null` (all pages) or an array of page keys the user may see. The app may use it to hide tabs, but it is optional.

4.2 GET /mobile-me — Current session

Returns the current user + tenant (fresh from DB). Use on app start to validate a stored token.

200 response

```

{
  "user": { "id": "0b3f...", "username": "admin", "role": "admin",
  "allowed_pages": null },
  "tenant": { "id": "a1c2...", "slug": "Cherif-Rent-Car", "name": "Cherif Rent
Car", "logo_url": "https://.../logo.png", "status": "active" }
}

```

Errors: 401 if token missing/expired or user deleted.

4.3 GET /mobile-cars — Fleet & availability

Returns all cars visible to the user with their **computed** status.

Query params | Param | Required | Description | |---|---|---| | `status` | no | Filter: `disponible`, `loue`, or `maintenance` |

200 response

```

{
  "cars": [
    {
      "id": 12,
      "brand": "Dacia",
      "model": "Logan",
      "plate_number": "123 TUN 4567",
      "status": "loue",
      "image_url": "https://.../car.jpg",
      "transmission": "manual",
      "mileage": 84000,
      "first_registration_year": 2021,
      "is_sous_traitance": false,
      "sous_traitance_name": null,
      "active_reservation": {
        "start_date": "2026-06-28T10:00:00.000Z",
        "end_date": "2026-07-02T10:00:00.000Z",
        "client_name": "Mohamed Ali",
        "contract_number": "CT-1024",
        "status": "active",

```

```

        "total_price": 600,
        "advance_payment": 200
      },
      "next_reservation": null
    }
  ],
  "counts": { "total": 1, "disponible": 0, "loue": 1, "maintenance": 0 }
}

```

Field notes

- **status**: disponible (free), loue (rented / out / subcontracted), maintenance (in immobilizing maintenance today).
- **active_reservation**: present only when the car is currently rented; null otherwise.
- **next_reservation**: the nearest upcoming booking ({ start_date, end_date }) or null.
- **is_sous_traitance / sous_traitance_name**: the car belongs to a subcontract.

4.4 GET /mobile-reservations — Reservations

Two modes:

A) List — GET /mobile-reservations

Param	Required	Default	Description
page	no	1	Page number (1-based)
pageSize	no	20	Items per page (max 100)
status	no	—	Filter: confirmed, active, completed, cancelled
search	no	—	Matches client name or reservation number

200 response

```

{
  "reservations": [
    {
      "id": 501,
      "reservation_number": "RES-17170000000000",
      "client_name": "Mohamed Ali",
      "client_phone": "+216 20 000 000",
      "car_id": 12,
      "start_date": "2026-06-28T10:00:00.000Z",
      "end_date": "2026-07-02T10:00:00.000Z",
      "status": "active",
      "total_price": 600,
      "advance_payment": 200,

```

```

    "contract_number": "CT-1024",
    "created_at": "2026-06-27T09:00:00.000Z",
    "car": { "brand": "Dacia", "model": "Logan", "license_plate": "123 TUN
4567", "image_url": "https://.../car.jpg" }
  },
  "total": 134,
  "page": 1,
  "pageSize": 20
}

```

Pagination: `total` is the full count for the filter. Compute pages as `ceil(total / pageSize)`.

B) Detail — GET /mobile-reservations?id=501

200 response

```

{
  "reservation": {
    "id": 501,
    "reservation_number": "RES-1717000000000",
    "client_name": "Mohamed Ali",
    "client_phone": "+216 20 000 000",
    "start_date": "2026-06-28T10:00:00.000Z",
    "end_date": "2026-07-02T10:00:00.000Z",
    "status": "active",
    "total_price": 600,
    "advance_payment": 200,
    "contract_number": "CT-1024",
    "car": { "id": 12, "brand": "Dacia", "model": "Logan", "license_plate": "123
TUN 4567", "image_url": "https://.../car.jpg" }
  }
}

```

Errors: 404 if the reservation doesn't exist or isn't visible to the user.

The detail response returns the full reservation row (*) plus the full car (car). Field availability depends on the DB schema; the keys above are the stable ones the app should rely on.

4.5 GET /mobile-vision360 — Dashboard / planning

Powers the "Vision360" screen: fleet (list view), a timeline of reservations, and a "returning soon" list.

Query params | Param | Required | Default | Description | |---|---|---|---| | `days` | no | 14 | Timeline window length in days (max 60) |

200 response

```

{
  "window": { "start": "2026-06-30T...", "end": "2026-07-14T...", "days": 14 },
  "cars": [ /* same shape as GET /mobile-cars cars[] */ ],
  "timeline": [
    { "id": 501, "car_id": 12, "start_date": "2026-06-28T10:00:00.000Z",
"end_date": "2026-07-02T10:00:00.000Z", "client_name": "Mohamed Ali", "status":
"active" }
  ]
}

```

```

    ],
    "returningSoon": [
      { "car_id": 12, "brand": "Dacia", "model": "Logan", "plate_number": "123 TUN
4567", "client_name": "Mohamed Ali", "end_date": "2026-07-02T10:00:00.000Z" }
    ]
  }
}

```

- **cars**: render the **list view** (each car + its current status).
- **timeline**: reservations overlapping the window — render the **chronology**.
- **returningSoon**: active rentals sorted by soonest return date.

4.6 GET /mobile-notifications — Notification feed

Computed feed of things needing attention (returns due/overdue + expiring papers). Call when the app opens / foregrounds, and to populate a bell screen.

200 response

```

{
  "notifications": [
    {
      "key": "return:501:2026-07-02T10:00:00.000Z",
      "type": "return_due",
      "title": "Retour à venir",
      "message": "Dacia Logan (Mohamed Ali) doit être rendue le 2026-07-02.",
      "date": "2026-07-02T10:00:00.000Z",
      "daysLeft": 2,
      "severity": "warning",
      "ref": { "reservation_id": 501, "car": "Dacia Logan" }
    },
    {
      "key": "VIGNETTE:88:2026-07-01",
      "type": "vignette",
      "title": "Vignette à renouveler",
      "message": "Vignette de Kia Picanto expire le 2026-07-01.",
      "date": "2026-07-01",
      "daysLeft": 1,
      "severity": "warning",
      "ref": { "maintenance_id": 88, "car": "Kia Picanto", "paper": "Vignette" }
    }
  ],
  "unread": 2,
  "generated_at": "2026-06-30T16:00:00.000Z"
}

```

Field notes

- **type**: return_due | vignette | assurance | visite_technique.
- **daysLeft**: days until date; **negative** = **overdue**.
- **severity**: info | warning | danger (use for color/icon).
- **key**: stable unique id — use it for de-duplication / "mark as read" locally.
- Sorted most-urgent first.

4.7 POST /mobile-notifications — Register push device

Register (or refresh) this device's push token so it can receive push.

Request body

```
{
  "push_token": "<device push token>",
  "platform": "android",
  "provider": "fcm"
}
```

Field	Required	Description
push_token	yes	The device token (FCM token for Flutter — see §5)
platform	no	android / ios (default expo)
provider	no	fcm / apns / expo (default expo)

200 response: { "ok": true }

Call this after login and whenever the push token refreshes.

4.8 DELETE /mobile-notifications — Unregister push device

Call on logout so the device stops receiving push.

Request body: { "push_token": "<device push token>" } **200 response:** { "ok": true }

5. Push notifications

Flow:

1. App obtains a device push token (Flutter → **Firebase Cloud Messaging**).
2. After login, app calls POST /mobile-notifications with that token.
3. Backend sends push for due/overdue returns and expiring papers on a schedule.
4. On logout, app calls DELETE /mobile-notifications.

Push payload data will include { type, key, ref } so the app can deep-link (e.g. open the matching reservation when type = return_due).

Backend coordination note (Flutter = FCM): the in-app feed (GET /mobile-notifications) works regardless of provider. The server-side **push delivery** is being aligned to **FCM** for Flutter. Confirm with the backend owner that FCM is configured before relying on background push; until then, use the feed endpoint (pull on

open/foreground), which is fully functional.

The app should still call GET /mobile-notifications on open/foreground to show an up-to-date list even when a push was missed.

6. Suggested screen → endpoint mapping

Screen	Endpoint(s)
Login	POST /mobile-auth
Splash / auto-login	GET /mobile-me
Fleet / cars grid	GET /mobile-cars (optionally ?status=)
Dashboard / planning	GET /mobile-vision360?days=14
Reservations list	GET /mobile-reservations? page=...
Reservation detail	GET /mobile-reservations?id=...
Notifications bell	GET /mobile-notifications
Enable push / logout	POST/DELETE /mobile-notifications

7. Flutter quick start (Dart http)

```
import 'dart:convert';
import 'package:http/http.dart' as http;

const baseUrl = 'https://<PROJECT_REF>.supabase.co/functions/v1';

class ApiClient {
  String? token;

  Map<String, String> _headers() => {
    'Content-Type': 'application/json',
    if (token != null) 'Authorization': 'Bearer $token',
  };

  Future<Map<String, dynamic>> login(
    String slug, String username, String password) async {
    final res = await http.post(
      Uri.parse('$baseUrl/mobile-auth'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'slug': slug, 'username': username, 'password':
password}),
    );
    if (res.statusCode != 200) {
      throw Exception(jsonDecode(res.body)['error'] ?? 'Login failed');
    }
    final data = jsonDecode(res.body) as Map<String, dynamic>;
    token = data['token'] as String;
    // TODO: persist token in flutter_secure_storage
  }
}
```



```

    return data;
  }

Future<Map<String, dynamic>> getCars({String? status}) async {
  final uri = Uri.parse('$baseUrl/mobile-cars')
    .replace(queryParameters: status != null ? {'status': status} : null);
  final res = await http.get(uri, headers: _headers());
  if (res.statusCode == 401) throw SessionExpired();
  if (res.statusCode != 200) {
    throw Exception(jsonDecode(res.body)['error'] ?? 'Request failed');
  }
  return jsonDecode(res.body) as Map<String, dynamic>;
}

Future<void> registerPush(String fcmToken) async {
  await http.post(
    Uri.parse('$baseUrl/mobile-notifications'),
    headers: _headers(),
    body: jsonEncode({
      'push_token': fcmToken,
      'platform': 'android', // or 'ios'
      'provider': 'fcm',
    })),
  );
}

class SessionExpired implements Exception {}

```

Recommended packages: http (or dio), flutter_secure_storage (token), firebase_messaging (push token).

8. Rules of thumb for the app

- Always send Authorization: Bearer <token> on every call except login.
 - On 401, clear the token and route to login.
 - This API is **read-only** — there are no endpoints to create/edit/cancel.
 - Don't hardcode <PROJECT_REF> or the tenant slug in source; make them configurable.
 - Poll GET /mobile-notifications on app open/foreground in addition to push.
-

API version: 1.0 (read-only). Questions or a missing field? Contact the backend owner.